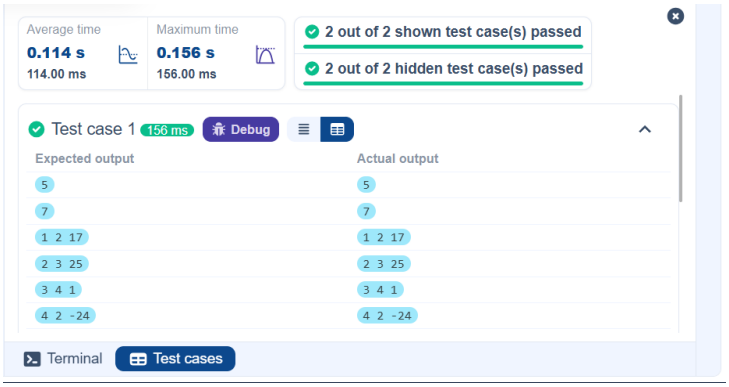
1.1.7

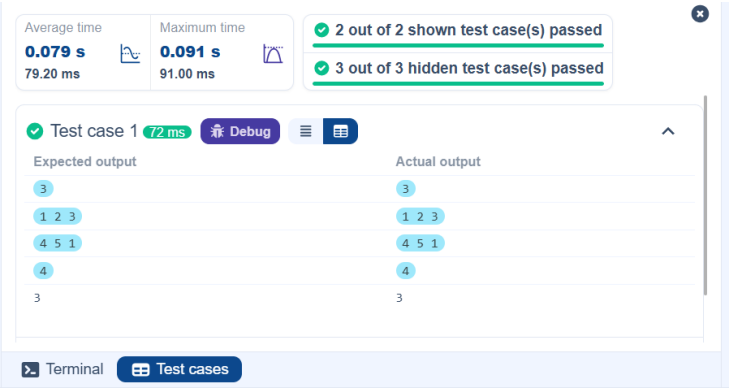
```
#include <stdio.h>
#include <limits.h>
#define INF LLONG_MAX
typedef long long ll;
typedef struct {
	int u, v;
	ll w;
} Edge;
int main() {
	int V, E;
	scanf("%d", &V);
	scanf("%d", &E);
	Edge edges[E];
	for (int i = 0; i < E; i++) {
		scanf("%d %d %lld", &edges[i].u, &edges[i].v, &edges[i].w);
	}
	int src;
	scanf("%d", &src);
	ll dist[V + 1];
	int parent[V + 1];
	for (int i = 1; i <= V; i++) {
		dist[i] = INF;
		parent[i] = -1;
	}
	dist[src] = 0;
	for (int i = 1; i <= V - 1; i++) {
		int updated = 0;
		for (int j = 0; j < E; j++) {
			int u = edges[j].u;
			int v = edges[j].v;
			ll w = edges[j].w;
			if (dist[u] != INF && dist[u] + w < dist[v]) {
				dist[v] = dist[u] + w;
				parent[v] = u;
				updated = 1;
			}
		}
		if (!updated)
			break;
	}
	for (int i = 0; i < E; i++) {
		int u = edges[i].u;
		int v = edges[i].v;
		ll w = edges[i].w;
		if (dist[u] != INF && dist[u] + w < dist[v]) {
			printf("Negative cycle detected\n");
			return 0;
		}
	}
	for (int v = 1; v <= V; v++) {
		if (v == src)
			continue;
		if (dist[v] == INF) {
```

```
                printf("%d INF None\n",
v);
                continue;
            }
            int path[V + 1];
            int count = 0;
            int current = v;
            while (current != -1) {
                path[count++] = current;
                if (current == src)
                    break;
                current =
parent[current];
            }
            printf("%d %lld ", v, dist[v]);
            if (path[count - 1] != src) {
                printf("None\n");
                continue;
            }
            for (int i = count - 1; i >= 0; i--) {
                printf("%d", path[i]);
                if (i > 0)
                    printf("->");
            }
            printf("\n");
        }
        return 0;
}
```



1.1.11

```
#include <stdio.h>
#include <stdlib.h>
int max(int a, int b) {
    return (a > b) ? a : b;
}
int main() {
    int N, W;
    if (scanf("%d", &N) != 1) return 0;
    int *values = (int *)malloc(N * sizeof(int));
    int *weights = (int *)malloc(N *
sizeof(int));
    for (int i = 0; i < N; i++) {
        if (scanf("%d", &values[i]) != 1)
return 0;
    }
    for (int i = 0; i < N; i++) {
        if (scanf("%d", &weights[i]) != 1)
return 0;
    }
    if (scanf("%d", &W) != 1) return 0;
    int *dp = (int *)calloc(W + 1, sizeof(int));
    for (int i = 0; i < N; i++) {
        for (int w = W; w >= weights[i]; w--
) {
            dp[w] = max(dp[w], dp[w
- weights[i]] + values[i]);
        }
    }
    printf("%d\n", dp[W]);
    free(values);
    free(weights);
    free(dp);
    return 0;
}
```



1.1.5

```
#include <stdio.h>
#include <stdlib.h>

// Global arrays to store discovered subsets for reverse printing
int **discovered_subsets;
int *subset_sizes;
int subset_count = 0;
int max_subsets = 1000; // Initial capacity for dynamic reallocation

// Backtracking function
void find_subsets(int *set, int n, int target_sum, int index, int current_sum, int *current_subset, int current_size) {
  // Base Case: If current sum matches the target, record the subset
  if (current_sum == target_sum) {
    // Expand storage capacity if needed
    if (subset_count >= max_subsets) {
      max_subsets *= 2;
      discovered_subsets = (int **)realloc(discovered_subsets, max_subsets * sizeof(int *));
      subset_sizes = (int *)realloc(subset_sizes, max_subsets * sizeof(int));
    }

    // Allocate space and copy the current subset
    discovered_subsets[subset_count] = (int *)malloc(current_size * sizeof(int));
    for (int i = 0; i < current_size; i++) {
      discovered_subsets[subset_count][i] = current_subset[i];
    }
    subset_sizes[subset_count] = current_size;
    subset_count++;
    return;
  }

  // If current sum exceeds target or we run out of elements, terminate this path
  if (current_sum > target_sum || index == n) {
    return;
  }

  // Choice 1: Include the current element
  current_subset[current_size] = set[index];
  find_subsets(set, n, target_sum, index + 1, current_sum + set[index], current_subset, current_size + 1);

  // Choice 2: Exclude the current element (Backtrack)
  find_subsets(set, n, target_sum, index + 1, current_sum, current_subset, current_size);
}

int main() {
  int n;
  if (scanf("%d", &n) != 1) return 0;
```

```
    int *set = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &set[i]) != 1) return 0;
    }

    int target_sum;
    if (scanf("%d", &target_sum) != 1) return 0;

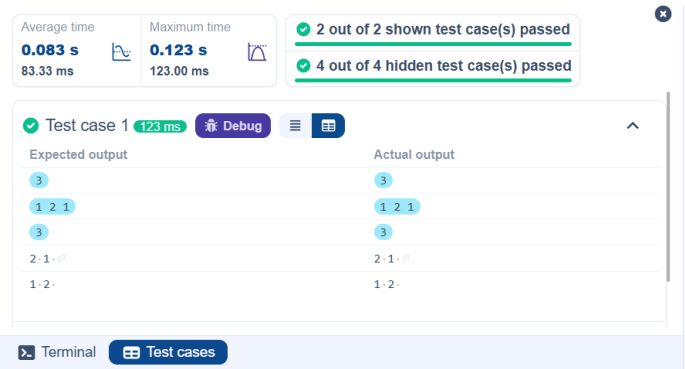
    // Allocate memory for tracking results
    discovered_subsets = (int **)malloc(max_subsets * sizeof(int *));
    subset_sizes = (int *)malloc(max_subsets * sizeof(int));
    int *current_subset = (int *)malloc(n * sizeof(int));

    // Run the backtracking algorithm
    find_subsets(set, n, target_sum, 0, 0, current_subset, 0);

    // Output formatting
    if (subset_count == 0) {
        printf("-1\n");
    } else {
        // Print in reverse order of discovery (last found -> printed first)
        for (int i = subset_count - 1; i >= 0; i--) {
            for (int j = 0; j < subset_sizes[i]; j++) {
                printf("%d", discovered_subsets[i][j]);
                if (j < subset_sizes[i] - 1) {
                    printf(" ");
                }
            }
            printf(" \n");
            free(discovered_subsets[i]); // Free individual subset memory
        }
    }

    // Clean up allocated memory
    free(discovered_subsets);
    free(subset_sizes);
    free(current_subset);
    free(set);

    return 0;
}
```



1.1.12

```
#include<stdio.h>

#include<string.h>

#include<stdbool.h>

#define MAX_LENGTH 100


int max(int a, int b){if(a>b){return a;}else{return b;}}


int lcsOf3(char X[], char Y[], char Z[], int m, int n,
int o){

        while(m > 0 && (X[m-1] == '\n' || X[m-1] ==
'\r')) m--;

        while(n > 0 && (Y[n-1] == '\n' || Y[n-1] ==
'\r')) n--;

        while(o > 0 && (Z[o-1] == '\n' || Z[o-1] ==
'\r')) o--;

        int dp[m + 1][n + 1][o + 1];


        for (int i = 0; i <= m; i++) {

                for (int j = 0; j <= n; j++) {

                        for (int k = 0; k <= o; k++) {

                                if (i == 0 || j == 0
|| k == 0) {


        dp[i][j][k] = 0;

                                }

                                else if (X[i - 1] ==
Y[j - 1] && X[i - 1] == Z[k - 1]) {


        dp[i][j][k] = dp[i - 1][j - 1][k - 1] + 1;

                                }

                                else {

        dp[i][j][k] = max(max(dp[i - 1][j][k], dp[i][j -
1][k]), dp[i][j][k - 1]);

                                }

                        }

                }

        }


        return dp[m][n][o];

}


int main()

{       char x[MAX_LENGTH],
y[MAX_LENGTH],z[MAX_LENGTH];

  fgets(x, MAX_LENGTH, stdin);

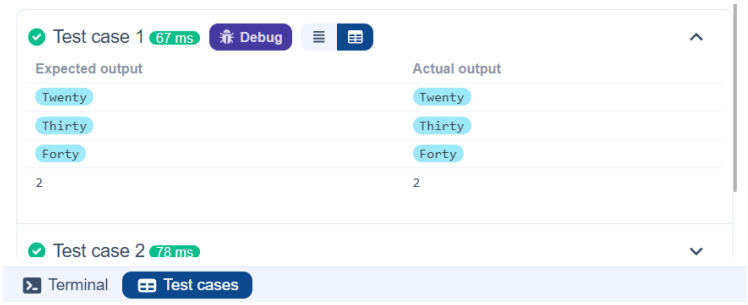
  fgets(y, MAX_LENGTH, stdin);

  fgets(z, MAX_LENGTH, stdin);

        printf("%d", lcsOf3(x, y, z, strlen(x),
strlen(y), strlen(z)));


        return 0;

}
```



1.1.13

```
#include <stdio.h>
#include <string.h>
#define INF 1000000000
int n;
int cost[20][20];
int memo[1 << 16][16];
int tsp(int mask, int u) {
    if (mask == (1 << n) - 1) {
        return (cost[u][0] != -1) ? cost[u][0] : INF;
    }
    if (memo[mask][u] != -1) {
        return memo[mask][u];
    }
    int minCost = INF;
    for (int v = 0; v < n; v++) {
        if (!(mask & (1 << v)) && cost[u][v] != -1) {
            int subCost = tsp(mask | (1 << v), v);
            if (subCost != INF) {
                int totalCost = cost[u][v] + subCost;
                if (totalCost < minCost) {
    minCost = totalCost;
                }
            }
        }
    }
    return memo[mask][u] = minCost;
}
int main() {
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }
    memset(memo, -1, sizeof(memo));
    int ans = tsp(1, 0);

    if (ans >= INF) {
        printf("-1\n");
    } else {
        printf("%d\n", ans);
    }
    return 0;
}
```

